The gates explained

What src/gates checks, with worked examples

Generated from docs/GATES_EXPLAINED.md, 2026-11-12

Contents

1. **The folder at a glance**
2. **The data gate:** `preflight.py`
3. **The serving gate:** `serve_check.py` **and** `serve_gate.py`
4. **If a gate fails**

A "gate" is a check that runs as a step of the pipeline and can stop it. There are two kinds: the **data gate** (is the data fit to train on?) and the **serving gate** (will the models fit on the Chisel server?). This guide explains the code in `src/gates/` with small examples. Every number in the examples was produced by running the real code.

1. The folder at a glance

File	What it is	Used by
`preflight.py`	the data gate: the checks themselves (domain shift, label balance)	`domain_check.py`, and optionally `train.py`
`production_shift.py`	does PRODUCTION look like the training data? A command you run on your machine, or `--prod-dir` on the pipeline launcher	you, and `src/orchestration/pipeline.py`
`domain_check.py`	the data gate as its own ClearML Task, the pipeline step `domain_check`	the pipeline
`serve_check.py`	the serving measurement: loads the ONNX models and times them like the server does	`serve_gate.py`, or by hand
`serve_gate.py`	the serving gate as a ClearML Task, the pipeline steps `serve_gate_pre` and `serve_gate_post`	the pipeline

The pattern is the same for both: a plain **checking** file (`preflight.py`, `serve_check.py`) that knows nothing about ClearML, and a thin **wrapper** (`domain_check.py`, `serve_gate.py`) that makes it a pipeline step.

How a gate stops a pipeline. Each check has a `mode` in its config:

- `gate`: a violation marks the step failed and exits with code 2 (`fail_gate` in `src/common/clearml_utils.py`). The steps after it are never created.
- `diagnostic`: the scores are reported in ClearML but nothing is stopped.

```
domain_check ---+
                +--> both must pass --> train_<class> (one per class) --> serve_gate_post
serve_gate_pre -+
```

2. The data gate: `preflight.py`

It compares two parts of your dataset, the **reference** (train) and the **candidate** (val), on a random sample of up to 500 tiles each (`conf/preflight/default.yaml`). It runs three checks.

Check	Question	Mode now	Limit
Domain shift	do val images look like train images?	`gate`	`max_shift` 0.5
Label balance	does each class take up a similar share of the tiles in both?	`diagnostic`	`max_wasserstein` 0.05, `max_ratio_gap` 0.5
Rarity	is any class too rare, in either split?	`diagnostic`	`min_tile_frac` 0.10, `min_mean_ratio` 0.01

The flow in the code: `gate_preflight` samples the tiles, then `run_preflight` runs `knn_interleaving` (domain shift), `class_rarity` and `label_balance` (both per class), writes `preflight.json`, and fails the Task if a check in `gate` mode is violated.

2.1 Domain shift: `knn_interleaving`

The idea. Describe every image by a few numbers. For each image, find its nearest neighbours, its "friends". If train and val look alike, a train image's friends should include plenty of val images. If val looks different, a train image's friends are all train images.

The numbers describing an image (`stats_features`): for each colour channel, the mean, the standard deviation, and the 5th and 95th percentile. For an RGB image that is 3 x 4 = 12 numbers.

A small example. To keep it readable, each image is described by one number, its brightness. There are 8 images: numbers 0 to 3 are train and numbers 4 to 7 are val, and we use `k = 2` friends each.

Case A: well mixed. Train brightness `[10, 20, 30, 40]`, val brightness `[15, 25, 35, 45]`.

```
image number:   0   1   2   3 | 4   5   6   7
pile (split):   0   0   0   0 | 1   1   1   1      0 = train, 1 = val
brightness:    10  20  30  40 | 15  25  35  45
```

Step 1, `neighbours`. One row per image: the **image numbers** of its 2 nearest friends (the image itself is removed).

```
image 0 (brightness 10): [4 1]   friends are image 4 (15) and image 1 (20)
image 1 (brightness 20): [5 4]   friends are image 5 (25) and image 4 (15)
image 2 (brightness 30): [5 6]
image 3 (brightness 40): [7 6]
image 4 (brightness 15): [0 1]
image 5 (brightness 25): [2 1]
image 6 (brightness 35): [2 3]
image 7 (brightness 45): [3 6]
```

Step 2, `split[neighbours]`. `split` is a lookup table: `split[4]` means "which pile is image number 4 in?". Each friend's image number is swapped for its pile.

```
image 0: friends [4 1] -> [split[4] split[1]] = [1 0]     friend 4 is val (1), friend 1 is train (0)
image 1: friends [5 4] -> [1 1]                            both friends are val
...
```

Step 3, `other`. For each image, the share of friends that are from the *other* pile than its own.

```
image 0 is train (0), its friends' piles are [1 0]  -> 1 of 2 differ = 0.5
image 1 is train (0), its friends' piles are [1 1]  -> 2 of 2 differ = 1.0
other = [0.5, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 0.5]        average 0.875
```

Step 4, `expected`**.** What `other` would be if the piles were mixed at random. For any image, 4 of the other 7 images are from the other pile: `4/7 = 0.571`.

Step 5, the score.

```
shift_score = clip( 1 − mean(other) / mean(expected), 0, 1 )
            = clip( 1 − 0.875 / 0.571, 0, 1 )  =  0
```

The strangers show up more often than by chance, so the piles are fully mixed and the score is 0.

Case B: partly separated. Train `[10, 20, 30, 40]`, val `[32, 42, 52, 62]`: val is brighter and overlaps only at the top.

```
image 0 (10): friends [1 2] -> piles [0 0]   both train      other = 0.0
image 2 (30): friends [4 1] -> piles [1 0]   one of each     other = 0.5   <- sits where the piles overlap
image 3 (40): friends [5 4] -> piles [1 1]   both val        other = 1.0   <- its nearest friends are val
image 7 (62): friends [6 5] -> piles [1 1]   both val        other = 0.0   <- own pile
other = [0, 0, 0.5, 1, 1, 0.5, 0, 0]       average 0.375

shift_score = 1 − 0.375 / 0.571 = 0.344
```

Friends come from the other pile only 0.375 of the time, below the 0.571 of a random mix, so the piles partly separate.

How to read the score.

Score	Meaning
0	train and val are mixed as much as chance, or more
about 0.3	partly separated: val looks somewhat different
close to 1	fully separated: val looks different from train
above `max_shift` (0.5)	the gate fails and the pipeline stops

The result also has `shift_score_ref` (looking only from the train images) and `shift_score_cand` (from the val images).

A real result. On OpenEarthMap, with val being 11 whole cities that training never saw, the gate gave **0.35** (train side 0.27, val side 0.43). That is about like case B: val looks somewhat different, as it should for different cities, and it passes the 0.5 limit.

A note on exact ties. The code removes the first friend of each image because it assumes that is the image itself. That holds unless two images have exactly the same description. With real colour statistics that practically never happens.

2.2 Label balance: `label_balance`

The idea. For one class (say road), measure how much of each tile it covers: its share of the tile's pixels. Then compare the list of shares in train with the list in val. If val has far less road, a model could look good or bad for the wrong reason.

It reports, per class:

Value	Meaning
`wasserstein`	how different the two lists of shares are (a distance: 0 means the same distribution)
`mean_ratio_ref`, `mean_ratio_cand`	the average share in train and in val
`ratio_gap_rel`	the relative gap between those two averages: `abs(a - b) / max(a, b)`
`empty_tile_frac_ref`, `_cand`	the share of tiles with none of this class

Example 1: similar. Road share per tile, six tiles each:

```
train: [0.00, 0.04, 0.06, 0.08, 0.10, 0.12]    average 0.067
val:   [0.00, 0.05, 0.06, 0.07, 0.10, 0.13]    average 0.068
wasserstein 0.005  (limit 0.05)    ratio_gap_rel 0.024  (limit 0.5)    -> PASS
```

Example 2: val has far less road.

```
train: [0.00, 0.04, 0.06, 0.08, 0.10, 0.12]    average 0.067,  1 of 6 tiles empty
val:   [0.00, 0.00, 0.01, 0.02, 0.02, 0.03]    average 0.013,  2 of 6 tiles empty
wasserstein 0.053  (over 0.05)    ratio_gap_rel 0.80  (over 0.5)    -> both limits violated
```

In `diagnostic` mode (the current setting) this is reported in ClearML but does not stop the pipeline.

A real result. On OpenEarthMap, the gate flagged only vegetation: wasserstein 0.070 against 0.05, because the held-out cities are greener. Building (0.049), road (0.011) and water (0.020) were under the limit.

2.3 Rarity: `class_rarity`

The idea. Label balance compares train with val. It cannot tell you that a class is rare in *both*. If water is in only 10% of the tiles, a model sees few examples and a validation score rests on a handful of tiles, so it swings from epoch to epoch. The rarity check looks at each class in each split on its own.

For every class and every split it reports:

Value	Meaning
`tiles_with`	how many of the sampled tiles contain at least one pixel of the class
`tile_frac`	that count as a share of the sampled tiles. Flagged below `min_tile_frac` (0.10)
`mean_ratio`	the class's average share of a tile's pixels. Flagged below `min_mean_ratio` (0.01)

Example. Water in 200 sampled val tiles:

```
train: 200 of 200 tiles contain water (100%), average 24% of a tile      -> fine
val:     6 of 200 tiles contain water (3%),   average 0.9% of a tile     -> flagged: 3% is below 10%, and 0.9% is
```

It is `diagnostic`: the report names the class and the split, and nothing is stopped. Set `mode: gate` in `conf/preflight/default.yaml` to stop the pipeline instead.

A real result. OpenEarthMap, 500 sampled tiles per split. No class is flagged:

Class	Train: tiles with the class, average share	Val: tiles with the class, average share
building	94%, 19.7%	91%, 14.8%
road	94%, 7.2%	86%, 6.2%
water	49%, 3.4%	45%, 5.1%
vegetation	99%, 26.5%	99%, 33.5%

Water is the rarest, and still in about half of the tiles. A tile counts if it has one pixel of the class, so a high `tile_frac` with a tiny `mean_ratio` can happen: that is why both numbers are checked.

2.4 Production shift: `production_shift.py`

The question. The data gate compares train with val, so it can only say that the dataset is consistent with itself. This check asks the question that matters for Chisel: *do the images the server will see look like the images the model learns from?* It uses the same neighbour test as section 2.1, with your production images on one side. They need no labels, and nothing is uploaded: it runs on your own machine.

```
python -m src.gates.production_shift --prod-dir <folder of production images> --root data/oem_416
```

Options: `--feature stats|encoder|both` (colour numbers, or the model's encoder features), `--n` tiles per side (300), `--per-image` windows from one image (8), `--tile` window size (416, the same as the training crops), `--out result.json`.

What it does:

1. Cuts 416 px windows from each production image. Transparent pixels (the area outside a drawn polygon) are never used: only windows fully inside the opaque area count. PNG, JPEG and TIFF all work.
2. Takes 300 random tiles of the training split.
3. Runs `knn_interleaving` on the colour numbers of both sets.
4. Does the same for the training split against its own val split, as a **baseline**. A production score of 0.7 means little unless you know that the baseline is 0.3.

A real result. 58 Singapore crops (55 had a usable window), against the OpenEarthMap train split:

Comparison	Shift score	Features
train vs val (other cities)	0.22	colour numbers
train vs Singapore production	**0.69**	colour numbers
train vs val	0.28	encoder

Comparison	Shift score	Features
train vs Singapore production	**0.70**	encoder

Both feature types agree: Singapore looks clearly different from the training cities, about two to three times as far as the training cities are from each other. Read it as "the model will see images unlike its training data", not as an accuracy figure. It does not tell you how well it will predict. For that you need labelled production crops.

Caveats. Windows cut from one image look alike, so the check warns when fewer than 100 independent images were used; with 55 images the production score is somewhat optimistic. It also looks only at colour, not at what is in the image.

On the pipeline launcher. The same check can run before a pipeline is created:

```
python -m src.orchestration.pipeline --config pipeline/pipeline_oem.yaml --local --queue kaggle \
    --prod-dir <folder> [--prod-root data/oem_416] [--prod-max-shift 0.5] [--prod-stop] ...
```

It prints the report, then launches as usual. With `--prod-stop` it refuses to launch (exit code 2, no ClearML task is created) when the production score is above `--prod-max-shift`. Without `--prod-dir` nothing changes.

2.5 Where to see the results

In the ClearML Task `domain_check`: the **Scalars** tab has `preflight/domain_shift`, `preflight/rarity` and `preflight/label_balance/<class>`, the **Plots** tab has histograms of the shares per class, and the **Artifacts** tab has `preflight` (the same as `preflight.json`).

3. The serving gate: `serve_check.py` and `serve_gate.py`

The question. Chisel runs all four models in one server process, with limits (about 4 to 8 CPUs and 8 GiB of memory). Can they run together inside those limits? It tests resources, not accuracy.

3.1 `serve_check.py`: the measuring tool

It loads every `.onnx` file in a folder into one process the way the server does, and runs requests through them:

- onnxruntime on CPU with the server's options (2 threads per model, memory arena off),
- a pool of 2 worker threads and one lock per model,
- for each tile and each model: RGB/255 -> run the model -> `postprocess_semantic_masks`.

It reports the time per request (mean and p95), peak memory, and how many CPU cores were busy. It fails (exit code 2) if peak memory is above `memory_limit_gib`, or if the p95 time is above `max_request_s` when that is set (it is `null` now).

A real example (this Mac, the four OpenEarthMap Unet models together, one 1280 px tile per request):

Measure	Result
time per request	4.66 s

Measure	Result
time per model	about 2.3 s each
peak memory	2.80 GiB
CPU cores busy	3.9
verdict	passed (limit 8 GiB)

The same check with four SegFormer b0 models took 2.62 s and 5.50 GiB, which is faster but twice the memory. This is why the gate exists: a model that looks small on disk (15 MB) can still use a lot of memory for its calculations.

Run it by hand: `python -m src.gates.serve_check --models <folder> --tiles 1`.

3.2 `serve_gate.py`: the pipeline step

It gets the models, runs `serve_check` in a fresh process (so downloading and exporting don't add to the memory reading), and reports the numbers to ClearML. `serve.source` in `conf/serve/default.yaml` picks the models:

`serve.source`	Models used	Step
`random`	untrained models built from `serve.spec`	`serve_gate_pre`, before training
`tasks`	the `onnx_shrunk` files of the `train_<class>` steps of the same pipeline run, found by the tag `pipe: <id>`	`serve_gate_post`, after training
`folder`	the `.onnx` files in `serve.folder`	local use

Untrained models are fine for the first run because speed and memory depend on the architecture and its size, not on what was learned.

3.3 Limits of this check

- It is only meaningful in a container that enforces the real limits. On a Kaggle agent or a Mac it reports a warning, "no CPU/memory cgroup limit detected", and the numbers are a rough guide.
- It uses random-noise tiles, so it says nothing about the quality of the predictions.
- It does not cover decoding the image, the alpha mask, choosing between classes, colouring or encoding the PNG that the real server does.

4. If a gate fails

What failed	Likely cause	What to do
`domain_check`: shift_score above 0.5	val looks too different from train (a region split with very different cities, or a bad split)	look at the score for the train and val side; change the split, or raise `max_shift` on purpose
`domain_check`: rarity flagged	a class is in too few tiles, or covers too few pixels, in one split	in `diagnostic` mode it doesn't stop. Pick a dataset or split where the class is more common, or accept a noisier score for it

What failed	Likely cause	What to do
`production_shift`: score above 0.5	production images look different from the training data (a different city, camera, season or format)	add training data that looks like production, or accept it knowingly. The score does not say how accurate the model is
`domain_check`: label balance violation	a class is much rarer or commoner in val	in `diagnostic` mode it doesn't stop; check whether the split is lopsided
`serve_gate_pre`: memory above the limit	the architecture is too big for the server	choose a smaller architecture or encoder before training
`serve_gate_post`: no ONNX model found	a `train_<class>` step has not completed, or has no `onnx` artifact	check that step first

The code that explains all of this: `src/gates/preflight.py` (checks), `src/gates/domain_check.py` and `src/gates/serve_gate.py` (steps), `src/gates/serve_check.py` (measurement), `conf/preflight/default.yaml` and `conf/serve/default.yaml` (limits).